Webアプリ開発実習プログラム

顧客管理 Web アプリケーションを題材に、要件定義から設計、実装、テスト、発表までを一通り経験する 1 か月・108 時間の実践型プログラムです。

総時間
108時間

実施日数
18日 × 6時間

対象
初級 - 中級

開発方式
0からの新規開発

React　PHP　SQLite　GitHub

PROGRAM PREMISE

小規模な開発案件として進める

この実習は「授業」ではなく、小さな開発プロジェクトとして進行します。要件を決め、設計し、作り、直し、説明するところまでを自分で回すことが前提です。

- 完成条件を先に決める
- 日次で Issue と README を更新する
- 説明可能な状態まで仕上げる

テーマ例: 顧客管理 Web アプリ / 固定スタック: React, PHP, SQLite, GitHub

この実習で体験すること

実装力だけでなく、仕事の進め方と説明力まで含めて鍛える。

PILLAR 01

セルフディレクション

- まず自分で調べる
- 論点を整理して仮説を立てる
- 次の一手を自分で決める

PILLAR 02

開発プロセスの通し経験

- 調査から発表までを順番に実施する
- 各工程ごとに成果物を残す
- 工程同士のつながりを理解する

PILLAR 03

実務に近い進め方

- Issue 管理を前提に進める
- feature ブランチと PR を使う
- レビューと README 更新を日常化する

重要なのは「たくさん作ること」ではなく、「完成条件を決め、品質を維持しながら仕上げること」です。

開発はこの順番で進める

各工程は前の工程の成果物を引き継ぎます。途中を飛ばさず、小さく区切って進めます。

1
調査
業務シナリオと課題を理解する

2
要件定義
MVP と受入条件を決める

3
設計
画面・DB・API を整合させる

4
実装
縦切りで一覧から順に作る

5
テスト
正常系と異常系を確認する

6
発表
成果物と判断理由を説明する

一覧表示を最初に通す「縦切り」方式で進めるため、フロントエンドと API の統合を最後まで後回しにしません。

週ごとの到達目標

毎週の目標と成果物を固定し、進捗を途中で見失わない構成にします。

WEEK 1

立ち上げ・要件定義・設計

- 要求を文書化する
- 一覧機能の最小縦切りを動かす
- 成果物: 要件定義書 v1 / 設計書 v1

WEEK 2

MVP 実装

- CRUD を UI から使える状態にする
- PR 記録と README 中間版を残す
- 成果物: MVP 一式

WEEK 3

テスト・品質改善

- テスト結果で品質を上げる
- 小規模な変更要求を反映する
- 成果物: テストシート / バグ一覧

WEEK 4

仕上げ・発表

- 成果物を説明可能な状態に整える
- 安定したデモを実施する
- 成果物: 最終 README / 発表資料

各週で成果物を残すため、最終週に説明資料や文書を慌てて作らない進め方にしています。

1日の標準リズムと相談ルール

日次の進め方を固定することで、実装・記録・相談の抜け漏れを防ぎます。

開始 - 30分	前日の残課題確認、当日の Issue 選定、作業計画の共有
午前帯	調査・設計・実装の主作業を進める。必要な調査結果はメモに残す。
午後前半	実装継続、テスト、レビュー対応、文書更新
終了前 - 30分	進捗共有、ブロッカー記録、README / Issue 更新、翌日の着手点決定

Rule 1

まず 30 分は自分で調査し、仮説を立てる

Rule 2

次に 15 分は周囲と相談し、切り分けを進める

Rule 3

止まる場合は、試したことと原因候補を添えて講師へ相談する

Rule 4

ブロッカーは Issue に記録し、口頭だけで終わらせない

自走とは「一人で抱え込むこと」ではありません。根拠を持って助けを求めることも実務スキルです。

成果物はコードだけではない

文書、開発プロセス、動く成果物、発表までを一式で揃えて提出します。

DOCUMENT

文書

- 要件定義書
- 設計書（構成図、DB、API、画面遷移）
- README
- 手動テストシート

PROCESS

開発プロセス

- Issue ボード
- feature ブランチと PR
- レビュー対応の履歴
- バグ一覧と修正記録

PRODUCT

成果物

- main ブランチで起動するソースコード
- デモシナリオ
- 変更要求への対応結果

PRESENTATION

発表

- 発表資料
- 最終デモ
- 振り返りシート

評価は完成度と進め方の両方を見る

機能数だけではなく、品質・再現性・説明力まで含めて評価します。

40%

成果物品質

MVP の完成度、安定性、変更要求への対応、デモ再現性

30%

開発プロセス

Issue、ブランチ、PR、レビュー、日次更新の実施状況

20%

文書・テスト

要件、設計、README、テストシートが実装と一致しているか

10%

発表

背景、判断、詰まり、学びを説明できるか

DO

やること

- MVP を先に完成させる
- 毎日 README と Issue を更新する
- 実装前に完了条件を確認する
- 詰まりは根拠付きで共有する

DON'T

やらないこと

- 技術スタックを勝手に変更する
- MVP 未完了のまま追加機能へ進む
- レビューなしで main に入れる
- 最終週に新機能を増やす